SDLC :

SDLC (Software Development Life Cycle) is a structured process used to develop, test, deploy, and maintain software.
In DevOps, SDLC is enhanced with automation, CI/CD pipelines, and continuous monitoring to deliver software faster and more reliably.

---

1. What is SDLC?

SDLC is the step-by-step process followed by software teams to build high-quality applications.

Full Form:
SDLC = Software Development Life Cycle

Goal of SDLC:

- Produce high-quality software
- Deliver software on time
- Reduce development cost
- Improve collaboration between teams

2. SDLC Phases

The SDLC process consists of **7 main phases**.

	Phase	Description	Tools Used in DevOps
1	**Planning**	Identify project goals and requirements	Jira, Trello
2	**Requirement Analysis**	Understand customer requirements	Confluence
3	**Design**	System architecture and design	UML, Draw.io
4	**Development**	Writing application code	Git, GitHub
5	**Testing**	Checking software for bugs	Selenium, JUnit
6	**Deployment**	Releasing software to production	Docker, Kubernetes
7	**Maintenance**	Fixing bugs and updates	Monitoring tools

3. SDLC Flow

Typical SDLC flow:

Planning → Requirement → Design → Development → Testing → Deployment → Maintenance

In **DevOps**, this process becomes **continuous** using automation.

---

4. SDLC in DevOps (Modern Approach)

Traditional SDLC was **slow and manual**, but DevOps improves it by using:

- **Continuous Integration (CI)** – Automatically build and test code
- **Continuous Delivery (CD)** – Automatically deploy applications
- **Automation** – Reduce manual work
- **Monitoring** – Track system performance

DevOps SDLC cycle:

Plan → Code → Build → Test → Release → Deploy → Operate → Monitor

---

5. DevOps Tools Used in Each SDLC Phase

SDLC Phase		DevOps Tool
8	Planning	Jira
9	Code	Git, GitHub
10	Build	Maven, Gradle
11	Test	Selenium
12	CI/CD	Jenkins
13	Container	Docker
14	Deployment	Kubernetes
15	Infrastructure	Terraform
16	Configuration	Ansible
17	Monitoring	Prometheus, Grafana

Since you are learning **Terraform and Ansible**, they are mainly used in:

- Infrastructure provisioning
- Configuration management
- Automated deployment

---

6. SDLC Models

Common SDLC models include:

1. Waterfall Model
2. Agile Model
3. Spiral Model
4. V-Model
5. DevOps Model

Today, most companies prefer **Agile + DevOps**.

---

7. Waterfall vs DevOps

Waterfall	DevOps
Sequential process	Continuous process
Slow development	Fast development
Manual deployment	Automated deployment
Less collaboration	High collaboration

---

8. Simple Definition (For Beginners)

SDLC is the complete process used to design, develop, test, deploy, and maintain software applications.

What is AWS?

Amazon Web Services (AWS) is a **cloud computing platform provided by Amazon**. It offers many services such as computing power, storage, databases, networking, and security through the internet.

In simple terms, AWS allows you to use servers, storage, and other IT resources online instead of buying and maintaining physical hardware.

Simple Definition

AWS is a cloud platform that provides on-demand IT resources like servers, storage, and databases over the internet with pay-as-you-go pricing.

Example

Instead of buying a physical server for a website:

1. Go to AWS
2. Launch a virtual server using **Amazon EC2**
3. Install your application
4. Access it from anywhere

So AWS acts like a **virtual data center in the cloud**.

Main AWS Service Categories

1. Compute

Provides virtual machines and processing power.
Example: **Amazon EC2**, **AWS Lambda**

2. Storage

Stores files, backups, and data.
Example: **Amazon S3**

3. Database

Managed database services.
Example: **Amazon RDS**

4. Networking

Creates networks and manages traffic.
Example: **Amazon VPC**

5. Security

Controls access and permissions.
Example: **AWS Identity and Access Management**

Key Features of AWS

- **Scalable** – Increase or decrease resources anytime
- **Pay-as-you-go** – Pay only for what you use
- **Highly available** – Data centers worldwide
- **Secure** – Strong security tools and encryption

Real-World Uses of AWS

Companies use AWS to:

- Host websites and applications
- Store large amounts of data
- Run AI and machine learning
- Build DevOps pipelines
- Backup and disaster recovery

Major companies using AWS include **Netflix**, **Airbnb**, and **Adobe**.

AWS Services

1. Compute Services

These services provide **computing power (servers)** in the cloud.

□ Amazon EC2

- EC2 = **Elastic Compute Cloud**
- Used to create **virtual servers in the cloud**
- You can install **Linux, Windows, applications**
- Pay only for the time you use

Uses

- Web hosting
- Application servers
- DevOps practice labs

Example

- Launch Ubuntu EC2 instance
- Install Nginx / Apache

□ AWS Lambda

- Run code **without managing servers**
- Event-based execution
- Supports **Python, Node.js, Java**

Example

- Run code when file uploaded to S3

□ Amazon Elastic Kubernetes Service

- Managed **Kubernetes** service
- Used to run **containers**

---

2. Storage Services

These services store **files, backups, databases**.

□ Amazon S3

- S3 = **Simple Storage Service**
- Used to store **files, images, videos, backups**
- Highly durable storage

Features

- Unlimited storage
- Versioning
- Lifecycle policies

□ Amazon EBS

- Elastic Block Store
- Used as **disk for EC2 instances**

Example:

- Attach 50GB disk to EC2 server

□ Amazon EFS

- Elastic File System
- Shared file system for multiple EC2 instances

---

3. Networking Services

These services control **network and connectivity**.

□ Amazon VPC

- Virtual Private Cloud
- Create **private network in AWS**

Components:

- Subnets
- Route Tables
- Internet Gateway
- NAT Gateway

□ Elastic Load Balancing

- Distributes traffic to multiple servers
- Improves **availability**

Types

- Application Load Balancer
- Network Load Balancer

□ Amazon Route 53

- Domain Name System service
- Maps domain → IP address

Example
example.com → EC2 instance

---

4. Database Services

□ Amazon RDS

- Managed database service
- Supports:
  - MySQL
  - PostgreSQL
  - Oracle
  - MariaDB

Benefits

- Automatic backups

- Scaling
- Security

□ Amazon DynamoDB

- Fully managed **NoSQL database**
- Very fast performance

---

5. Security Services

□ AWS Identity and Access Management

- Manage **users and permissions**
- Control access to AWS resources

Concepts

- Users
- Groups
- Roles
- Policies

□ AWS Shield

- Protects from **DDoS attacks**

---

6. DevOps Services

□ AWS CodePipeline

- Continuous Integration & Deployment

□ AWS CodeBuild

- Build and test code automatically

□ AWS CodeDeploy

- Deploy applications automatically

---

7. Monitoring Services

□ Amazon CloudWatch

- Monitor AWS resources
- Collect logs and metrics

Example

- Monitor EC2 CPU usage

Simple AWS Architecture Example

```
User
  ↓
Route53 (DNS)
  ↓
Load Balancer
  ↓
EC2 Instances
  ↓
RDS Database
  ↓
S3 Storage
```

GitHub:

GitHub is a web-based platform used to store, manage, and share source code. It uses Git (a version control system) to track changes in files.

Simple Definition

- **Git** → Tool used to track changes in code.
- **GitHub** → Online platform where Git repositories are stored and shared.

Why GitHub is used

- Store project code online

- Collaborate with developers
- Track changes in files
- Maintain version history
- Backup code safely

Example

If you develop an application, you upload the code to GitHub so other developers can **view, edit, and contribute**.

GitHub Commands and Their Uses in DevOps

GitHub is widely used in DevOps to store code, collaborate, and integrate with CI/CD pipelines. It works with **Git** for version control.

In **DevOps**, GitHub is used to:

- Store **application code**
- Store **infrastructure code (Terraform, Ansible, etc.)**
- Track changes
- Enable **team collaboration**
- Trigger **CI/CD pipelines**

---

1. Git Configuration Commands

`git config`

Used to configure Git settings.

```
git config --global user.name "YourName"
git config --global user.email "your@email.com"
```

Use in DevOps:
Used when setting up Git on a new **server, VM, or developer machine**.

---

2. Repository Commands

`git init`

Creates a new repository.

```
git init
```

Use in DevOps:
Initialize repository for:

- Terraform infrastructure
- Ansible playbooks
- Application source code

`git clone`

Downloads repository from GitHub.

```
git clone repository_url
```

Example:

```
git clone https://github.com/user/project.git
```

Use in DevOps:
Developers and servers clone the repo to run or deploy the application.

---

3. File Management Commands

`git status`

Shows current repository status.

```
git status
```

Use in DevOps:
Check which files were modified before committing changes.

`git add`

Adds files to staging area.

```
git add filename
```

Add all files:

```
git add .
```

Use in DevOps:
Stage configuration files like:

- `main.tf`
- `ansible.yml`
- `Dockerfile`

---

4. Commit Commands

`git commit`

Saves changes to the repository.

```
git commit -m "Added Terraform VPC configuration"
```

Use in DevOps:
Track infrastructure changes and maintain version history.

Example commits:

- Updated Jenkins pipeline
- Added new EC2 configuration
- Fixed Ansible playbook

---

5. Remote Repository Commands

`git remote add`

Connect local repository to GitHub.

```
git remote add origin repository_url
```

Example:

```
git remote add origin https://github.com/user/devops-project.git
```

Use in DevOps:
Links local infrastructure code to GitHub repository.

---

6. Push Command

`git push`

Uploads code to GitHub.

```
git push origin main
```

Use in DevOps:
Push code to trigger:

- CI/CD pipeline
- Jenkins builds
- Automated deployments

---

7. Pull Command

`git pull`

Downloads latest code and merges it.

```
git pull origin main
```

Use in DevOps:
Developers update their code before working.

---

8. Fetch Command

`git fetch`

Downloads updates without merging.

```
git fetch
```

Use in DevOps:
Check remote changes before merging.

---

9. Branch Commands

`git branch`

Lists branches.

```
git branch
```

Create branch:

```
git branch feature-login
```

Use in DevOps:
Developers work on **separate features without affecting production code**.

`git checkout`

Switch branch.

```
git checkout dev
```

Create and switch branch:

```
git checkout -b new-feature
```

Use in DevOps:
Used in **feature development and testing environments**.

---

10. Merge Command

`git merge`

Combines branches.

```
git merge dev
```

Use in DevOps:
Merge feature branch into:

- development branch
- main branch

---

11. Log Command

`git log`

Shows commit history.

```
git log
```

Use in DevOps:
Check previous changes and rollback if needed.

---

12. Reset Command

`git reset`

Undo staged changes.

```
git reset filename
```

Use in DevOps:
Fix mistakes before committing code.

---

13. Stash Command

`git stash`

Temporarily saves changes.

```
git stash
```

Use in DevOps:
Switch branches without losing work.

GitHub Workflow in DevOps

Typical DevOps workflow:

```
Developer writes code
        ↓
git add .
        ↓
git commit
        ↓
git push
        ↓
GitHub Repository
        ↓
CI/CD Pipeline Trigger
        ↓
Build → Test → Deploy
```